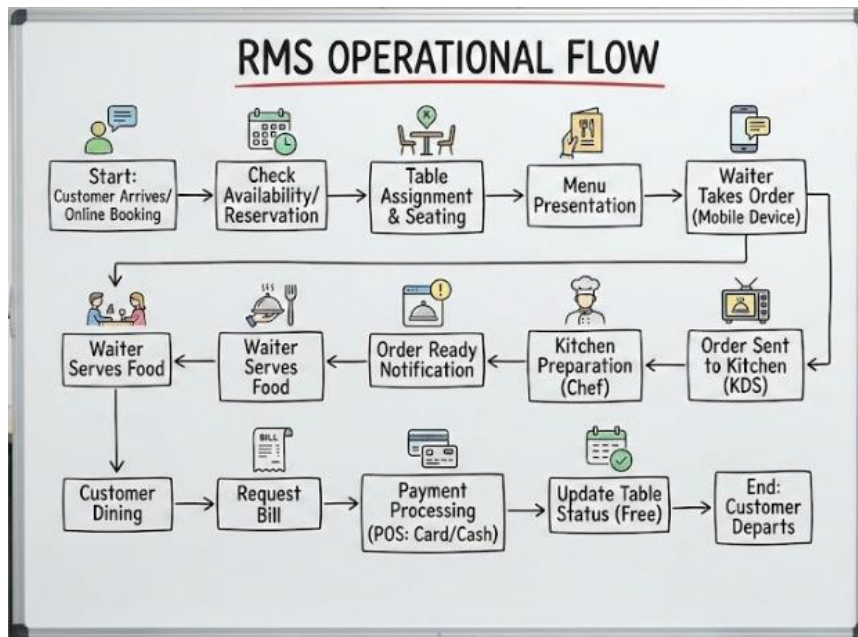


Restaurant Management System (RMS)

Overview



A restaurant management system (RMS) is a type of software that helps people who run restaurants manage different parts of their business. This includes tracking sales of food and drinks, organizing employee schedules, and handling customer reservations. It can be used in various kinds of eateries such as fast-food joints, fancy restaurants, and cafeterias. With this system, restaurants can work more smoothly, become more efficient, and offer better service to their customers.

Main Functions

The RMS allows restaurant managers to oversee and manage their operations from one place. They can check which tables are free, book reservations, and create bills.

Purpose

The main purpose of the RMS is to make various tasks easier and more automated for restaurant owners and managers. This helps them concentrate on providing a great dining experience for their guests.

Interview Preparation

Here are some aspects you might need to discuss in more detail during an interview:

Restaurant Services

- Does the restaurant offer delivery?
- Can customers order online?
- Does the restaurant accept online or card payments?

Restaurant Management

- Can services vary between different branches of the restaurant?
- Should we consider how the restaurant manages its stock of items?

Design Approach

We plan to design this system using a bottom-up approach, which means:

1. Start by designing the smallest parts like tables, seats, meals, meal items, and the seating layout.
2. Use these to design larger components like the menu, each branch, and the whole restaurant.
3. Keep building on these until the entire system is designed.

Design Pattern

Discussing the design patterns used in the RMS during an interview shows you understand advanced design concepts. This makes a good impression.

Requirements for Restaurant Management System

Here are the main requirements for the RMS:

1. The restaurant offers a menu with different sections and items.
2. Waiters can create orders for a table and add items for each person seated.
3. Each person's order can include multiple menu items.
4. The system should show available tables for walk-in customers.
5. The system should allow for table reservations.
6. The receptionist should be able to search for available tables by date and time and make reservations.
7. The system should allow customers to make and cancel reservations.
8. The system should send reminders as the reservation time gets close.
9. Customers should be able to pay with credit cards, checks, or cash.
10. Each branch may have different table setups.

This is a classic Object-Oriented Design (OOD) and System Design interview question. Since you mentioned a **bottom-up approach**, the goal is to define the core entities first (the "nouns") and then build the workflows (the "verbs") on top of them.

Here is a step-by-step guide on how to navigate this interview, from the first interaction to the final design.

Phase 1: Clarification & Scoping (The "What" and "Why")

Before drawing anything, you must clarify constraints. This shows you care about requirements before coding.

How to introduce this:

"Before we dive into the design, I'd like to clarify a few functional and non-functional requirements to ensure we are building the right system."

Questions you should ask the interviewer:

1. Scope of the System:

- *"Is this a system for a single local restaurant, a chain of restaurants (like McDonald's), or a SaaS platform (like OpenTable) used by thousands of different restaurants?"*
(Assume a chain/branch model based on your prompt).

2. Concurrency & Scale:

- *"How many concurrent users are we expecting? Is the focus on handling high traffic (scalability) or complex logic (data consistency for reservations)?"*

3. Specific Features:

- *"Do we need to handle staff shift management, or just focus on reservations and orders?"*
- *"For payments, do we process them internally or just integrate with a gateway (like Stripe/PayPal)?"*

Phase 2: Core Entities (Bottom-Up Design)

As per your prompt, we start small. In an interview, you would list these out on the whiteboard.

1. The Atoms (Smallest Units)

- **MealItem:** The basic food unit (e.g., "Cheese Burger", "Coke"). It needs a name, description, and price.
- **Table:** Needs a unique ID, capacity (how many seats), and status (Free, Reserved, Occupied).
- **Seat:** (Optional, depending on granularity) useful if splitting checks by seat.

2. The Composites (Building Up)

- **Menu:** Contains a list of `MenuSection` (Appetizers, Mains), which contain `MealItems`.
- **Order:** Belongs to a specific `Table` and `Waiter`. Contains a list of `OrderItems`.
- **Reservation:** Links a `Customer` to a `Table` for a specific `TimeSlot`.

3. The Macro Level

- **Branch/Location:** Has a physical address and a specific set of `Tables`.
- **Restaurant:** The parent entity that owns multiple `Branches`.

Phase 3: Database Schema & Relationships

Now, map these entities to a database. For an RMS, a **Relational Database (SQL)** (like PostgreSQL or MySQL) is usually preferred over NoSQL because strict data consistency (ACID) is required. You cannot have two people booking the same table at the same time.

Key Tables:

- **Restaurants:** `id, name`
- **Branches:** `id, restaurant_id, address, phone`
- **Tables:** `id, branch_id, table_number, capacity, status`
- **Menu:** `id, branch_id, active_date`
- **Reservations:**
 - `id`
 - `table_id` (Foreign Key)
 - `customer_id` (Foreign Key)
 - `reservation_time`
 - `status` (PENDING, CONFIRMED, CANCELLED)
- **Orders:** `id, table_id, status, total_amount`
- **Order_Items:** `id, order_id, meal_item_id, quantity, price`

Phase 4: API Design (The Interface)

Explain to the interviewer how the outside world communicates with your system. RESTful APIs are the standard here.

Core Endpoints:

1. Reservation Search:

- `GET /api/v1/branches/{branchId}/tables?date={date}&time={time}&capacity={count}`
- *Response:* List of available tables.

2. Make Reservation:

- `POST /api/v1/reservations`
- *Body:* `{ customerId, tableId, time, partySize }`

3. Place Order:

- `POST /api/v1/orders`
- *Body:* `{ tableId, waiterId, items: [{itemId, qty}, ...] }`

4. Checkout/Pay:

- `POST /api/v1/orders/{orderId}/payment`

Phase 5: The Critical Flow (Handling Concurrency)

This is where you pass or fail the "Senior" level check. You must explain how you prevent **Double Booking**.

The Scenario:

Two customers, Alice and Bob, see Table #5 is free at 7:00 PM. They both click "Book" at the exact same millisecond.

The Solution: Database Locking

You can use **Optimistic Locking**.

1. The `Tables` table has a `version` number.
2. Alice reads Table #5 (Version 1).
3. Bob reads Table #5 (Version 1).
4. Alice's transaction tries to update:
 - `UPDATE Tables SET status='Reserved', version=2 WHERE id=5 AND version=1;`
 - **Result:** Success (1 row affected).
5. Bob's transaction tries to update:
 - `UPDATE Tables SET status='Reserved', version=2 WHERE id=5 AND version=1;`
 - **Result:** Fail (0 rows affected) because the version is now 2, not 1.
6. The system tells Bob, "Sorry, this table was just taken."

Phase 6: Notifications & Payments (Wrapping up)

1. Notifications:

- Use a message queue (like RabbitMQ or Kafka) or a simple background worker.
- When a reservation is created, publish an event: `RESERVATION_CONFIRMED` .
- A notification service consumes this event and sends an SMS/Email to the user.
- *Feature:* A scheduled job runs every hour to check for upcoming reservations (e.g., "reservation_time - 1 hour") and triggers a reminder.

2. Payments:

- Do not store credit card numbers.
- Pass the payment details directly to a provider (Stripe/PayPal) and only store the `transaction_id` and `status` (PAID/FAILED) in your database.

Summary Checklist for the Interview

1. **Clarify:** Ask about scale and multi-branch support.
2. **Define Entities:** Start with Table/Meal, end with Restaurant.
3. **Schema:** Choose SQL for consistency.
4. **Concurrency:** Mention Optimistic Locking for reservations.
5. **Design Patterns:** Mention specific patterns if you know them (e.g., **State Pattern** for handling Order Status: `Open` -> `In Progress` -> `Served` -> `Paid`).

2. How to Explain This to the Interviewer

When presenting this design, you want to guide the interviewer through the logic rather than just dropping classes on a board.

Step 1: Start with the base structures

"I started with a bottom-up approach. The foundational data structures here are the `Seat` and the `MealItem`. I've used composition (the solid diamond relationship) to indicate that `Seats` cannot exist without a `Table`, and a `Menu` cannot exist without a `Branch`."

Step 2: Explain Actor Abstraction

"For the users of the system, I created an abstract `Person` class. From there, `Employee` inherits common fields like name and email, but adds employee-specific IDs. `Waiter` and `Receptionist` then inherit from `Employee`. This adheres to the **Open/Closed Principle**, allowing us to easily add a `Manager` or `Chef` role later without modifying existing code."

Step 3: Detail the Core Workflows

"The two main workflows are Ordering and Reservations. An `Order` aggregates multiple `OrderItems`, which hold the state of the price at the time of purchase—this is critical so that if a `MealItem` price changes tomorrow, historical orders aren't affected."

3. High-Value Questions to Ask the Interviewer

Interviews are collaborative. Asking the right clarifying questions mid-design shows seniority and foresight. Here are a few to keep in your back pocket:

- **Concurrency & State Management:** "If a `Waiter` and a `Customer` attempt to update an `Order` at the exact same time, should we enforce optimistic locking on the database level for the `Order` entity?"
- **Design Pattern Implementation:** "For the `Payment` process, I've set it up as an `Interface` to utilize the **Strategy Pattern**. This allows us to inject `CreditCardPayment` or `CashPayment` dynamically. Should we flesh out the external gateway integration for the credit cards?"
- **Edge Cases:** "How strict are our table turnover rules? For example, if a `Reservation` is at 8:00 PM, should the `Table` lock out walk-in customers starting at 6:30 PM to ensure it's empty?"
- **Scaling Data:** "As historical `Orders` and `Reservations` pile up, query speeds for `getAvailableTables()` might slow down. Would you like me to discuss how we might archive old records or use a caching layer like Redis for today's table availability?"

For a **Restaurant Management System (RMS)**, here are the **5 most critical Design Patterns** you should mention, exactly *where* to apply them, and *why* they are necessary.

1. State Pattern (Crucial for Orders & Tables)

This is the most impressive pattern to discuss because it solves the complex logic of "changing statuses" without using messy `if-else` statements.

- **Where to use it:**
 - **Order Lifecycle:** An order goes through strict stages: `Received` → `Preparing` → `Ready` → `Served` → `Paid`.
 - **Table Lifecycle:** A table transitions from `Free` → `Reserved` → `Occupied` → `Free`.
- **Why use it:**
 - **Prevents Invalid Actions:** You cannot "Serve" an order that hasn't been "Prepared." You cannot "Reserve" a table that is currently "Occupied."
 - **Code Cleanliness:** Instead of a giant `switch` statement checking the current status, each state is its own class.
- **Example:**
 - If you call `cancel()` on a `PaidOrderState`, the class simply throws an error or ignores it.
 - If you call `cancel()` on a `ReceivedOrderState`, it processes the refund logic.

2. Strategy Pattern (Crucial for Payments)

This pattern is the standard answer for anything involving "algorithms that can change at runtime."

- **Where to use it:**
 - **Payment Processing:** A customer might pay via `CreditCard`, `Cash`, `PayPal`, or `ApplePay`.
 - **Pricing Strategies:** Different calculation logic for `RegularHours`, `HappyHour` (20% off), or `HolidayPricing`.
- **Why use it:**
 - **Open/Closed Principle:** You can add a new payment method (e.g., "Bitcoin") *without* modifying the existing `Order` class. You just create a new `BitcoinStrategy` class.
 - **Runtime Switching:** The user selects "Credit Card" on the UI, and the system dynamically injects the `CreditCardStrategy`.
- **Code Concept:**

3. Observer Pattern (Crucial for Notifications)

Restaurants rely on real-time communication. The kitchen needs to talk to the waiter, and the system needs to talk to the customer.

- **Where to use it:**
 - **Order Status Updates:** When the Chef marks an order as `Ready`, the Waiter (and potentially the Customer's app) needs to be notified immediately.
 - **Reservation Alerts:** When a table becomes free, customers on the waiting list could be notified.
- **Why use it:**
 - **Decoupling:** The `Order` object doesn't need to know *who* is listening. It just broadcasts "I am ready!" and any registered listener (Waiter App, Kitchen Display, SMS Service) reacts accordingly.
 - **Scalability:** You can add new listeners (e.g., a "Manager Dashboard") without changing the core Order logic.

4. Composite Pattern (Crucial for the Menu)

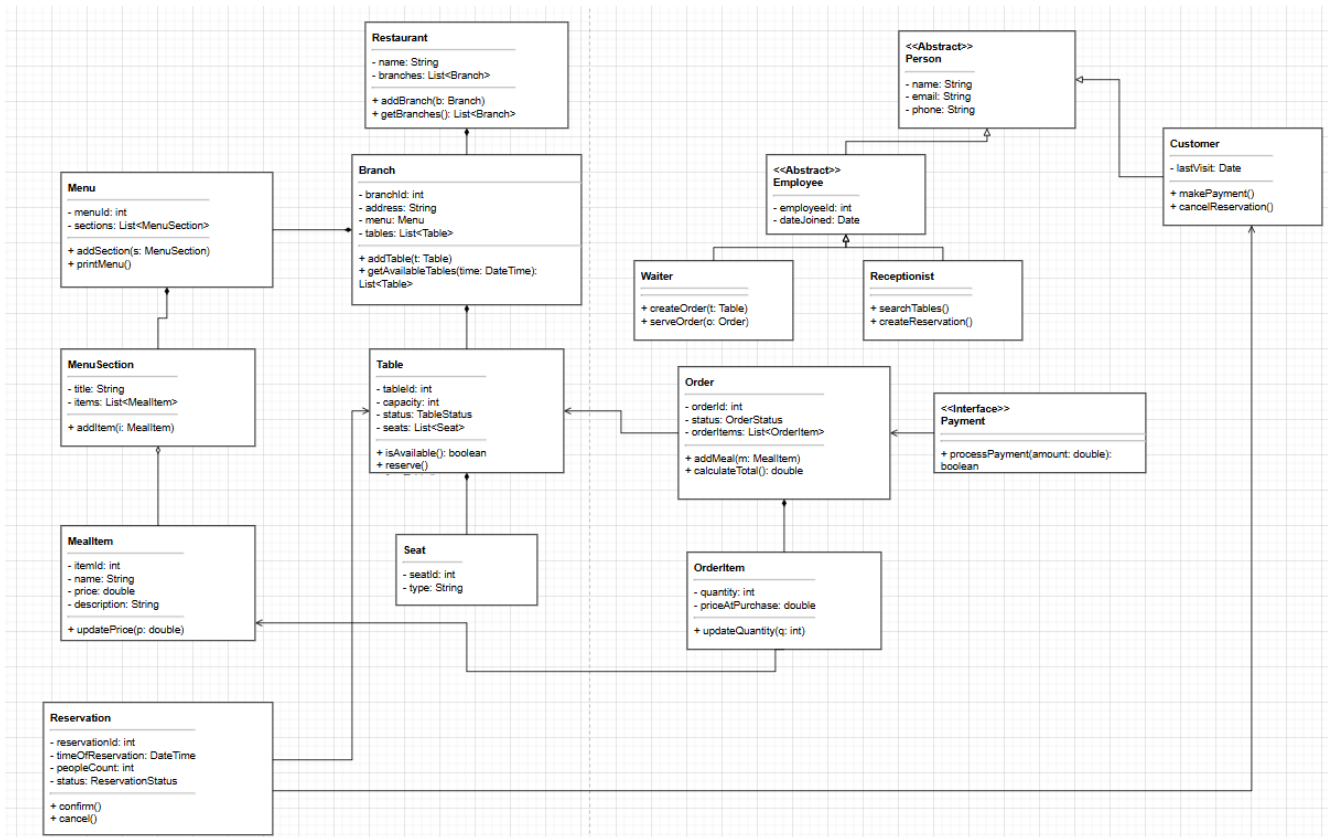
This pattern is used to treat individual objects and groups of objects in the same way.

- **Where to use it:**
 - **The Menu Structure:** A `Menu` contains `Sections` (e.g., Lunch, Dinner), which contain `Categories` (Appetizers, Mains), which contain `Items` (Burger, Fries).
 - Sometimes an "Item" is a combo (Burger + Fries).
- **Why use it:**
 - **Uniformity:** You can treat a single "Cheese Burger" and a "Combo Meal" (which contains a burger and fries) exactly the same.
 - **Recursive Operations:** If you want to `print()` the menu, you call it on the top level, and it cascades down through all sections and items automatically.

5. Singleton Pattern (Infrastructure)

- **Where to use it:**
 - **Database Connection:** To ensure we don't open 1,000 connections for 1,000 orders.
 - **Configuration Manager:** To load settings like "Restaurant Name" or "Tax Rate" once and access them globally.
- **Why use it:**
 - **Resource Management:** Ensures there is exactly one shared instance of a heavy resource.

Class Diagram



Restaurant Management System API

1.0.0 OAS 3.0

API for managing restaurant operations including table reservations, menu management, order processing, and payments.

[Contact API Support](#)

Servers

https://api.restaurant.com/v1 - Production Server

Reservations

Managing table bookings and availability

- GET

/branches/{branchId}/tables/search

Search for available tables
- POST

/reservations

Create a new reservation
- PUT

/reservations/{reservationId}/cancel

Cancel a reservation

Orders

Kitchen order management and lifecycle

- POST

/orders

Create a new order for a table
- GET

/orders/{orderId}

Get order details
- PUT

/orders/{orderId}

Update order status or items

Menu

Menu items and categories

- GET

/branches/{branchId}/menu

Get the active menu for a branch

Payments

Payment processing

- POST

/orders/{orderId}/payment

Process payment for an order

Why Clean Architecture?

1. **Framework Independent:** The core business logic doesn't depend on Spring Boot, .NET, or any specific database.
2. **Testable:** You can test the business rules without spinning up a database or web server.
3. **Interchangeable:** If you want to swap the Payment Gateway from Stripe to PayPal, you only change the "Infrastructure" layer. The core logic remains untouched.

```
src/
├── config/                # Global app configuration (Swagger, CORS, Security)
│   ├── OpenApiConfig.java
│   └── SecurityConfig.java
├── core/                  # THE HEART: Pure Business Logic (No frameworks allowed)
│   ├── domain/           # The "Nouns" (Entities)
│   │   ├── model/
│   │   │   ├── Table.java
│   │   │   ├── Reservation.java
│   │   │   ├── Order.java
│   │   │   └── User.java
│   │   └── enums/
│   │       ├── TableStatus.java
│   │       └── PaymentStatus.java
│   ├── usecase/          # The "Verbs" (Business Actions / Interfaces)
│   │   ├── ports/        # Interfaces defining what we need (Dependency Inverted)
│   │   │   ├── incoming/ # API calls this (e.g., CreateReservationUseCase)
│   │   │   ├── outgoing/ # Data Access (e.g., ReservationRepository)
│   │   │   └── services/  # The actual logic implementation
│   │       ├── ReservationService.java (Implements CreateReservationUseCase)
│   │       └── OrderService.java
│   └── infrastructure/   # THE IMPLEMENTATION: Database, External APIs
│       ├── persistence/  # Database interaction
│       │   ├── entity/   # DB specific entities (JPA/Hibernate/EF Core)
│       │   ├── repository/ # Implementation of 'outgoing' ports
│       │   │   ├── JpaReservationRepository.java
│       │   │   └── SqlOrderRepository.java
│       │   └── mapper/    # Converts DB Entity <-> Domain Model
│       ├── external/     # Third-party integrations
│       │   ├── payment/
│       │   │   ├── StripePaymentGateway.java
│       │   │   └── PayPalPaymentGateway.java
│       │   └── notification/
│       │       └── EmailService.java
│       └── presentation/ # THE ENTRY POINT: REST Controllers
│           ├── api/
│           │   ├── v1/
│           │   │   ├── ReservationController.java
│           │   │   └── OrderController.java
│           │   └── dto/    # Data Transfer Objects (What the API sends/receives)
│           │       ├── request/
│           │       │   └── CreateReservationRequest.java
│           │       └── response/
│           │           └── ReservationResponse.java
│           ├── exception/ # Global Exception Handling
│           │   └── GlobalExceptionHandler.java
│           └── AppApplication.java # Main entry point
```

Detailed Breakdown for the Interview

When the interviewer asks, *"Walk me through your code structure,"* you should explain it layer by layer, starting from the inside out.

1. The Core (Inner Layer)

- **Folder:** `core/domain` & `core/usecase`
- **Purpose:** This contains the enterprise business rules.
- **Key Rule:** This layer has **ZERO dependencies**. It does not know about SQL, HTTP, or JSON. It only contains plain objects (POJOs/POCOs).
- **Why:** This ensures your logic is pure and won't break if you upgrade your database version.

2. The Ports (The Glue)

- **Folder:** `core/usecase/ports`
- **Purpose:** Since the Core can't talk to the Database directly (that would be a dependency), we define **Interfaces** here.
- **Example:** `ReservationRepository` is an interface with a method `save(Reservation r)`. The Core calls this interface, not caring *how* it's saved.

3. Infrastructure (The Outer Layer)

- **Folder:** `infrastructure/`
- **Purpose:** This is where the dirty work happens. This layer implements the interfaces defined in the Core.
- **Example:** `JpaReservationRepository` implements the `ReservationRepository` interface and uses SQL to save data.
- **Why:** If we switch from MySQL to MongoDB, we only rewrite this file. The Core logic remains 100% the same.

4. Presentation (The Edge)

- **Folder:** `presentation/`
- **Purpose:** This handles the HTTP requests. It converts JSON into Java/C# objects and calls the Core services.
- **Key Rule:** Controllers should be "dumb." They shouldn't contain business logic (e.g., checking if a table is free). They just pass data to the Service.